Name: Aryan Sharma
Reg. No.: 23BCE1320
Subject: Cryptography & Network Security Lab

1. Diffie–Hellman Key Exchange
Diffie–Hellman allows two parties to create a shared secret over an insecure channel without directly sending the key.
Steps:
1. Publicly agree on prime **p** and generator **g**.
2. Alice selects private key **a**, Bob selects private key **b**.
3. Compute public keys:
   - A = g^a mod p
   - B = g^b mod p
4. Exchange public keys.
5. Compute shared secret:
   - Alice: S = B^a mod p
   - Bob: S = A^b mod p

Both obtain the same secret key **S**, used for encryption.

2. Multi-Party Communication Using Sockets
- A server manages communication.
- Three clients connect: **Alice**, **Bob**, and **Attacker**.
- The server relays public keys and encrypted messages.

Each client:
- Generates private/public keys
- Computes shared secret
- Encrypts and decrypts messages

3. Man-in-the-Middle (MITM) Attack
Without authentication, Diffie–Hellman is vulnerable to MITM attacks.
Process:
1. Alice and Bob send public keys.
2. Attacker intercepts them.
3. Attacker sends fake keys to both parties.

Result:
- Alice shares a key with attacker.
- Bob shares a key with attacker.
- Alice and Bob do not share a key.

The attacker can intercept, modify, and forward messages without detection.

Code:
Client.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class Client {

    public static void main(String[] args) throws Exception {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter role (alice/bob/attacker): ");
        String r = sc.nextLine();

        Socket s = new Socket("localhost", 5000);

        BufferedReader in = new BufferedReader(
                new InputStreamReader(s.getInputStream()));
        PrintWriter out = new PrintWriter(
                s.getOutputStream(), true);

        String[] pms = in.readLine().split(",");
        long p = Long.parseLong(pms[0]);
        long g = Long.parseLong(pms[1]);

        if (r.equalsIgnoreCase("attacker")) {

            long prA = HellmanUtils.generatePrivateKey();
            long prB = HellmanUtils.generatePrivateKey();

            long puA = HellmanUtils.generatePublicKey(g, prA, p);
            long puB = HellmanUtils.generatePublicKey(g, prB, p);

            long realAPub = Long.parseLong(in.readLine());
            long realBPub = Long.parseLong(in.readLine());

            out.println(puA);
            out.println(puB);

            long keyWithAlice = HellmanUtils.computeSharedSecret(realAPub,
prA, p);
            long keyWithBob = HellmanUtils.computeSharedSecret(realBPub, prB,
p);

            System.out.println("Key with Alice: " + keyWithAlice);
            System.out.println("Key with Bob: " + keyWithBob);

            while (true) {

                String packet = in.readLine();

                if (packet.startsWith("A:")) {

                    String enc = packet.substring(2);
                    System.out.println("\nEncrypted from Alice: " + enc);
```

```
            String dec = HellmanUtils.decrypt(enc, keyWithAlice);
            System.out.println("Decrypted: " + dec);

            String mod = "[HACKED] " + dec;
            System.out.println("Modified: " + mod);

            String reEnc = HellmanUtils.encrypt(mod, keyWithBob);
            System.out.println("Sent to Bob: " + reEnc);

            out.println(reEnc);
        }

        else if (packet.startsWith("B:")) {

            String enc = packet.substring(2);
            System.out.println("\nEncrypted from Bob: " + enc);

            String dec = HellmanUtils.decrypt(enc, keyWithBob);
            System.out.println("Decrypted: " + dec);

            String mod = "[HACKED] " + dec;
            System.out.println("Modified: " + mod);

            String reEnc = HellmanUtils.encrypt(mod, keyWithAlice);
            System.out.println("Sent to Alice: " + reEnc);

            out.println(reEnc);
        }
    }
}

else {

    long pr = HellmanUtils.generatePrivateKey();
    long pu = HellmanUtils.generatePublicKey(g, pr, p);

    out.println(pu);

    long rcv = Long.parseLong(in.readLine());
    long key = HellmanUtils.computeSharedSecret(rcv, pr, p);

    System.out.println("Shared key: " + key);

    try {

        while (true) {

            System.out.print("Enter message: ");
            String msg = sc.nextLine();

            String enc = HellmanUtils.encrypt(msg, key);
            out.println(enc);

            String rep = in.readLine();
            String dec = HellmanUtils.decrypt(rep, key);

            System.out.println("Received: " + dec);
        }

    } finally {
```

```
                s.close();
                sc.close();
            }
        }
    }
}
```

Server.java

```
import java.io.*;
import java.net.*;

public class Server {

    private static final int PORT = 5000;
    private static final long P = 23;
    private static final long G = 5;

    public static void main(String[] args) throws Exception {

        ServerSocket ss = new ServerSocket(PORT);
        System.out.println("Server started. Waiting for connections...");

        Socket a = ss.accept();
        System.out.println("Alice connected");

        Socket b = ss.accept();
        System.out.println("Bob connected");

        Socket m = ss.accept();
        System.out.println("Attacker connected");

        BufferedReader inA = new BufferedReader(new
InputStreamReader(a.getInputStream()));
        PrintWriter outA = new PrintWriter(a.getOutputStream(), true);

        BufferedReader inB = new BufferedReader(new
InputStreamReader(b.getInputStream()));
        PrintWriter outB = new PrintWriter(b.getOutputStream(), true);

        BufferedReader inM = new BufferedReader(new
InputStreamReader(m.getInputStream()));
        PrintWriter outM = new PrintWriter(m.getOutputStream(), true);

        String params = P + "," + G;
        outA.println(params);
        outB.println(params);
        outM.println(params);

        long aPub = Long.parseLong(inA.readLine());
        long bPub = Long.parseLong(inB.readLine());

        outM.println(aPub);
        outM.println(bPub);
```

```
        long fakeA = Long.parseLong(inM.readLine());
        long fakeB = Long.parseLong(inM.readLine());

        outA.println(fakeA);
        outB.println(fakeB);

        System.out.println("MITM established. Public keys replaced.");

        while (true) {

            String msgA = inA.readLine();
            outM.println("A:" + msgA);
            String modA = inM.readLine();
            outB.println(modA);

            String msgB = inB.readLine();
            outM.println("B:" + msgB);
            String modB = inM.readLine();
            outA.println(modB);
        }
    }
}
```

CryptoUtils.java

```
import java.util.Random;

public class HellmanUtils {

    public static long modExp(long b, long e, long m) {

        long r = 1;
        b = b % m;

        while (e > 0) {

            if ((e & 1) == 1) {
                r = (r * b) % m;
            }

            e = e >> 1;
            b = (b * b) % m;
        }

        return r;
    }

    public static long generatePrivateKey() {
        Random rd = new Random();
```

```
        return rd.nextInt(100) + 1;
    }

    public static long generatePublicKey(long g, long pr, long p) {
        return modExp(g, pr, p);
    }

    public static long computeSharedSecret(long pub, long pr, long p) {
        return modExp(pub, pr, p);
    }

    public static String encrypt(String msg, long k) {

        StringBuilder sb = new StringBuilder();
        int sh = (int) (k % 26);

        for (char c : msg.toCharArray()) {

            if (Character.isLetter(c)) {
                char base = Character.isUpperCase(c) ? 'A' : 'a';
                sb.append((char) ((c - base + sh) % 26 + base));
            } else {
                sb.append(c);
            }
        }

        return sb.toString();
    }

    public static String decrypt(String msg, long k) {

        StringBuilder sb = new StringBuilder();
        int sh = (int) (k % 26);

        for (char c : msg.toCharArray()) {

            if (Character.isLetter(c)) {
                char base = Character.isUpperCase(c) ? 'A' : 'a';
                sb.append((char) ((c - base - sh + 26) % 26 + base));
            } else {
                sb.append(c);
            }
        }

        return sb.toString();
    }
}
```

Output
Server:

```
◆ lab5 ⚡ main ⊙
> java Server
Server started. Waiting for connections...
Alice connected
Bob connected
Attacker connected
MITM established. Public keys replaced.
```

Alice

```
◆ lab5 ⚡ main ⊙
> java Client
Enter role (alice/bob/attacker): alice
Shared key: 4
Enter message: Aryan
Received: [HACKED] Sharma
Enter message: Meet me after the toga party
◆ lab5 ⚡ main ⊙
> pwd

Path
----
C:\Users\aryan\Workspace\EDU\semester-6\cryptography\cryptography-network-security-lab\lab5
```

Bob

```
◆ lab5 ⚡ main ⊙
> java Client
Enter role (alice/bob/attacker): bob
Shared key: 4
Enter message: Sharma
Received: [HACKED] Aryan
Enter message: Exception in thread "main" java.util.NoSuchElementException: No line found
        at java.base/java.util.Scanner.nextLine(Scanner.java:1690)
        at Client.main(Client.java:102)
◆ lab5 ⚡ main ⊙
> pwd

Path
----
C:\Users\aryan\Workspace\EDU\semester-6\cryptography\cryptography-network-security-lab\lab5
```

Attacker

```
◆ lab5 ⚡ main ⊙
> java Client
Enter role (alice/bob/attacker): attacker
Key with Alice: 4
Key with Bob: 4

Encrypted from Alice: Evcer
Decrypted: Aryan
Modified: [HACKED] Aryan
Sent to Bob: [LEGOIH] Evcer

Encrypted from Bob: Wlevqe
Decrypted: Sharma
Modified: [HACKED] Sharma
Sent to Alice: [LEGOIH] Wlevqe

Encrypted from Alice: Qiix qi ejxiv xli xske tevxc
Decrypted: Meet me after the toga party
Modified: [HACKED] Meet me after the toga party
Sent to Bob: [LEGOIH] Qiix qi ejxiv xli xske tevxc
```

5. Observations

- Alice and Bob generated different shared keys due to the attacker.
- Communication occurred through the attacker without their knowledge.
- Messages were successfully intercepted and modified.
- Diffie–Hellman works correctly but is vulnerable without authentication.
- Socket programming allowed real-time simulation of the attack.

6. Security Recommendations

1. Use authentication methods (digital signatures, certificates).
2. Use secure protocols such as TLS.
3. Verify public keys before accepting them.
4. Implement proper identity verification.
5. Avoid unauthenticated key exchange in real systems.

7. Conclusion

This experiment demonstrated how Diffie–Hellman establishes a shared secret over an insecure channel and how it can be exploited using a Man-in-the-

Middle attack. It highlighted the importance of authentication in key exchange protocols. The lab also improved understanding of socket programming and showed that encryption alone is not sufficient without identity verification.